

```

"""
app_support.py
=====

Plumbing for the Streamlit app: configuration bridging, access control,
persistence, and serialisation.

`bootstrap()` MUST run before anything imports `config`, because config.py
reads environment variables at import time and Streamlit Cloud supplies
configuration through `st.secrets` rather than a `.env` file.
"""

from __future__ import annotations

import json
import os
from dataclasses import replace
from datetime import datetime
from pathlib import Path
from typing import Any, Optional

import streamlit as st

APP_DIR = Path(__file__).resolve().parent
DATA_DIR = APP_DIR / "data"
WORKSPACE_FILE = DATA_DIR / "workspace.json"

# -----
# Bootstrap
# -----

def bootstrap() -> None:
    """Copy uppercase Streamlit secrets into os.environ, then prepare dirs.

    python-dotenv does not override existing environment variables, so values
    set here win over any local .env file. That is what you want on a hosted
    deployment.
    """
    DATA_DIR.mkdir(exist_ok=True)
    (APP_DIR / "output").mkdir(exist_ok=True)
    (APP_DIR / "logs").mkdir(exist_ok=True)

    try:
        secrets: dict[str, Any] = dict(st.secrets)

```

```

except Exception: # no secrets.toml at all -- fine for local runs
    secrets = {}

for key, value in secrets.items():
    if key.isupper() and isinstance(value, (str, int, float, bool)):
        os.environ[key] = str(value)

def secret(name: str, default: Optional[str] = None) -> Optional[str]:
    try:
        if name in st.secrets:
            return str(st.secrets[name])
    except Exception:
        pass
    return os.environ.get(name, default)

# -----
# Access control
# -----

def require_access() -> bool:
    """Gate the app behind a password.

    A hosted Streamlit URL is effectively public. Without a gate, anyone who
    finds it can spend your Anthropic credits and generate letters of intent
    carrying your company name and signature block. So the default is closed:
    set APP_PASSWORD in secrets, or deliberately set ALLOW_UNPROTECTED="true"
    if you are only running this on your own machine.
    """
    expected = secret("APP_PASSWORD")
    unprotected = (secret("ALLOW_UNPROTECTED", "") or "").lower() in ("1", "true")

    if not expected:
        if unprotected:
            st.warning(
                "This app has no password. Anyone with the link can use it and "
                "spend your API credits. Set APP_PASSWORD in your app secrets."
            )
            return True
    st.header("Set a password first")
    st.markdown(
        "This app is not protected yet, so it will not open.\n\n"
        "***To fix:** open your app on share.streamlit.io, go to "
        "***Settings -> Secrets**, and add a line:\n\n"
        "```\nAPP_PASSWORD = \"choose-something-long\"\n```\n\n"
        "Then reload this page.\n\n"

```

```

        "If you are running on your own computer and want to skip this, "
        "add `ALLOW_UNPROTECTED = \"true\"` instead."
    )
    return False

if st.session_state.get("_authenticated"):
    return True

st.header("Sign in")
entered = st.text_input("Password", type="password", key="_pw")
if st.button("Open the app", type="primary", use_container_width=True):
    if entered == expected:
        st.session_state["_authenticated"] = True
        st.rerun()
    else:
        st.error("That password did not match. Try again.")
return False

# -----
# Serialisation
# -----

def listing_to_dict(listing) -> dict[str, Any]:
    return listing.to_dict()

def listing_from_dict(data: dict[str, Any]):
    from models import Listing, ListingAgent, ListingSource, Provenance

    agent_data = data.get("agent") or {}
    try:
        agent_src = Provenance(agent_data.get("source", "missing"))
    except ValueError:
        agent_src = Provenance.MISSING

    agent = ListingAgent(
        name=agent_data.get("name"),
        email=agent_data.get("email"),
        phone=agent_data.get("phone"),
        brokerage=agent_data.get("brokerage"),
        source=agent_src,
    )

    try:
        source = ListingSource(data.get("source", "unknown"))
    except ValueError:

```

```

        source = ListingSource.UNKNOWN

    listing = Listing(
        listing_id=data.get("listing_id", ""),
        address=data.get("address", ""),
        city=data.get("city", ""),
        state=data.get("state", "OR"),
        zip_code=data.get("zip_code"),
        county=data.get("county"),
        list_price=data.get("list_price"),
        bedrooms=data.get("bedrooms"),
        bathrooms=data.get("bathrooms"),
        square_feet=data.get("square_feet"),
        lot_size_sqft=data.get("lot_size_sqft"),
        year_built=data.get("year_built"),
        stories=data.get("stories"),
        days_on_market=data.get("days_on_market"),
        price_reduction_count=data.get("price_reduction_count", 0),
        original_price=data.get("original_price"),
        motivation_keywords_found=list(data.get("motivation_keywords_found", []))
        description=data.get("description", ""),
        listing_url=data.get("listing_url"),
        agent=agent,
        source=source,
        source_message_id=data.get("source_message_id"),
        field_provenance=dict(data.get("field_provenance", {})),
        raw_text=data.get("raw_text", "")
    )
    return listing

def assessment_from_dict(data: Optional[dict[str, Any]]):
    if not data:
        return None
    from models import AFHAssessment

    return AFHAssessment(
        afh_feasibility_score=int(data.get("afh_feasibility_score", 0)),
        pros=list(data.get("pros", [])),
        red_flags=list(data.get("red_flags", [])),
        master_bedroom_shared_potential=bool(data.get("master_bedroom_shared_pote
        master_bedroom_reasoning=data.get("master_bedroom_reasoning", ""),
        ground_floor_bedroom_estimate=data.get("ground_floor_bedroom_estimate"),
        provider_room_viable=bool(data.get("provider_room_viable", False)),
        accessibility_notes=list(data.get("accessibility_notes", [])),
        zoning_hoa_risk_notes=list(data.get("zoning_hoa_risk_notes", [])),
        estimated_market_rent=data.get("estimated_market_rent"),

```

```

        suggested_phase1_holding_rent=data.get("suggested_phase1_holding_rent"),
        suggested_phase2_full_rent=data.get("suggested_phase2_full_rent"),
        rent_basis=data.get("rent_basis", ""),
        unknowns=list(data.get("unknowns", [])),
        model_used=data.get("model_used", ""),
    )

# -----
# Workspace persistence
# -----

EMPTY_WORKSPACE: dict[str, Any] = {
    "version": 1,
    "saved_at": None,
    "settings": {},
    "properties": [],
}

def load_workspace() -> dict[str, Any]:
    """Read the saved workspace from disk.

    On Streamlit Community Cloud this file lives on an ephemeral disk and is
    wiped whenever the app restarts or redeloys. Use the Back up button on the
    Settings tab to keep a copy in your iPad Files app.
    """
    if not WORKSPACE_FILE.exists():
        return json.loads(json.dumps(EMPTY_WORKSPACE))
    try:
        return json.loads(WORKSPACE_FILE.read_text(encoding="utf-8"))
    except Exception:
        return json.loads(json.dumps(EMPTY_WORKSPACE))

def save_workspace(workspace: dict[str, Any]) -> None:
    workspace["saved_at"] = datetime.now().isoformat(timespec="seconds")
    DATA_DIR.mkdir(exist_ok=True)
    WORKSPACE_FILE.write_text(json.dumps(workspace, indent=2), encoding="utf-8")

def workspace() -> dict[str, Any]:
    """Session-cached workspace."""
    if "workspace" not in st.session_state:
        st.session_state["workspace"] = load_workspace()
    return st.session_state["workspace"]

```

```

def persist() -> None:
    save_workspace(st.session_state["workspace"])

# -----
# Settings -> dataclass overrides
# -----

def company_from_settings(settings: dict[str, Any]):
    import config

    return replace(
        config.COMPANY,
        legal_name=settings.get("legal_name", config.COMPANY.legal_name),
        dba=settings.get("dba", config.COMPANY.dba),
        signer_name=settings.get("signer_name", config.COMPANY.signer_name),
        signer_title=settings.get("signer_title", config.COMPANY.signer_title),
        phone=settings.get("phone", config.COMPANY.phone),
        email=settings.get("email", config.COMPANY.email),
        mailing_address=settings.get("mailing_address", config.COMPANY.mailing_ad
    )

def assumptions_from_settings(settings: dict[str, Any]):
    import config

    def _n(key: str) -> Optional[float]:
        raw = settings.get(key)
        if raw in (None, "", 0):
            return None
        try:
            return float(raw)
        except (TypeError, ValueError):
            return None

    def _i(key: str) -> Optional[int]:
        val = _n(key)
        return int(val) if val is not None else None

    return replace(
        config.ASSUMPTIONS,
        resident_capacity=int(settings.get("resident_capacity", 5) or 5),
        medicaid_resident_count=_i("medicaid_resident_count"),
        medicaid_monthly_rate=_n("medicaid_monthly_rate"),
        medicaid_rate_source=settings.get("medicaid_rate_source", ""),
        private_pay_resident_count=_i("private_pay_resident_count"),

```

```

        private_pay_monthly_rate=_n("private_pay_monthly_rate"),
        private_pay_rate_source=settings.get("private_pay_rate_source", ""),
        room_and_board_monthly=_n("room_and_board_monthly"),
        room_and_board_source=settings.get("room_and_board_source", ""),
        staffing_cost=_n("staffing_cost"),
        food_supplies_cost=_n("food_supplies_cost"),
        insurance_cost=_n("insurance_cost"),
        utilities_cost=_n("utilities_cost"),
        assumptions_effective_date=settings.get("assumptions_effective_date", "")
    )

def thresholds_from_settings(settings: dict[str, Any]):
    import config

    return replace(
        config.THRESHOLDS,
        min_bedrooms=float(settings.get("min_bedrooms", 5) or 5),
        min_days_on_market=int(settings.get("min_dom", 30) or 0),
        max_list_price=int(settings.get("max_list_price", 900_000) or 900_000),
        min_list_price=int(settings.get("min_list_price", 200_000) or 0),
        go_no_go_score=int(settings.get("go_no_go_score", 75) or 75),
    )

# -----
# Gmail from secrets (hosted OAuth workaround)
# -----

def gmail_available() -> bool:
    return bool(secret("GMAIL_TOKEN_JSON"))

@st.cache_resource(show_spinner=False)
def gmail_service():
    """Build a Gmail service from a token stored in secrets.

    The desktop OAuth flow in gmail_ingestion.py opens a loopback browser,
    which cannot work on a hosted server. Instead you run that flow ONCE on a
    computer, then paste the resulting token.json contents into the app's
    secrets as GMAIL_TOKEN_JSON. See DEPLOY_IPAD.md.
    """
    raw = secret("GMAIL_TOKEN_JSON")
    if not raw:
        return None

    import config

```

```

from google.auth.transport.requests import Request
from google.oauth2.credentials import Credentials
from googleapiclient.discovery import build

creds = Credentials.from_authorized_user_info(json.loads(raw), config.GMAIL_S
if creds.expired and creds.refresh_token:
    creds.refresh(Request())
return build("gmail", "v1", credentials=creds, cache_discovery=False)

# -----
# Display helpers
# -----

def verdict(score: Optional[int], go_threshold: int) -> tuple[str, str, str]:
    """Return (band, label, hex). Bands are GREEN / YELLOW / RED / BLUE."""
    if score is None:
        return "BLUE", "Not scored yet", "#1F3A5F"
    if score >= go_threshold:
        return "GREEN", f"Go · {score}/100", "#1E7A4B"
    if score >= go_threshold - 15:
        return "YELLOW", f"Review · {score}/100", "#9A6B00"
    return "RED", f"No · {score}/100", "#A32C2C"

def money(value: Optional[float]) -> str:
    if value in (None, 0):
        return "-"
    return f"${value:,.0f}"

def mailto_link(to: str, subject: str, body: str) -> str:
    from urllib.parse import quote
    return f"mailto:{quote(to)}?subject={quote(subject)}&body={quote(body)}"

```